Python Code

```
# CARE TRANSITION DASHBOARD (CLEAN + MODERN)

import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

st.set_page_config(page_title="Care Analytics", layout="wide")

# LOAD DATA

@st.cache_data
def load_data():
    df = pd.read_csv("HHS_Unaccompanied_Alien_Children_Program.csv")

    df['Date'] = pd.to_datetime(df['Date'])

    df.columns = [
        "date",
        "cbp_intake",
        "cbp_custody",
        "cbp_transfer",
        "hhs_care",
        "hhs_discharge"
    ]

    # Fix numeric columns
    cols = df.columns[1:]
    for col in cols:
        df[col] = df[col].astype(str).str.replace(',', '')
        df[col] = pd.to_numeric(df[col], errors='coerce')

    df[cols] = df[cols].fillna(0)

    return df

df = load_data()


# KPIs

df['transfer_efficiency'] = df['cbp_transfer'] / df['cbp_custody'].replace(0, 1)
df['discharge_effectiveness'] = df['hhs_discharge'] / df['hhs_care'].replace(0, 1)
df['throughput'] = df['hhs_discharge'] / df['cbp_intake'].replace(0, 1)
df['backlog'] = df['cbp_intake'] - df['hhs_discharge']
df['backlog_ma7'] = df['backlog'].rolling(7).mean()


# SIDEBAR
```

```python

st.sidebar.title("Filters")

start = st.sidebar.date_input("Start", df['date'].min())
end = st.sidebar.date_input("End", df['date'].max())

filtered = df[(df['date'] >= pd.to_datetime(start)) &
              (df['date'] <= pd.to_datetime(end))]


# TITLE

st.markdown("## 📊 Care Transition Dashboard")
st.caption("Monitoring efficiency of CBP → HHS → Placement pipeline")

st.markdown("---")


# KPI CARDS (BETTER LOOK)

c1, c2, c3, c4 = st.columns(4)

c1.metric("Transfer Efficiency",
          f"{filtered['transfer_efficiency'].mean():.2f}")

c2.metric("Discharge Effectiveness",
          f"{filtered['discharge_effectiveness'].mean():.2f}")

c3.metric("Throughput",
          f"{filtered['throughput'].mean():.2f}")

c4.metric("Avg Backlog",
          int(filtered['backlog'].mean()))

st.markdown("---")


# CHART FUNCTION (REUSABLE)

def style_date_axis(ax):
    ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    plt.xticks(rotation=30)



# ROW 1 (FLOW)

col1, col2 = st.columns(2)

with col1:
    st.subheader("Pipeline Flow")

    fig, ax = plt.subplots()
```

```
    ax.plot(filtered['date'], filtered['cbp_intake'], label="Intake")
    ax.plot(filtered['date'], filtered['hhs_discharge'], label="Discharge")

    style_date_axis(ax)

    ax.legend()
    st.pyplot(fig)

with col2:
    st.subheader("Backlog Trend")

    fig, ax = plt.subplots()
    ax.plot(filtered['date'], filtered['backlog'], label="Backlog")
    ax.plot(filtered['date'], filtered['backlog_ma7'], linestyle='--')

    style_date_axis(ax)

    ax.axhline(0)
    st.pyplot(fig)


# ROW 2 (EFFICIENCY)

st.subheader("Efficiency Trends")

fig, ax = plt.subplots()

ax.plot(filtered['date'], filtered['transfer_efficiency'], label="Transfer")
ax.plot(filtered['date'], filtered['discharge_effectiveness'], label="Discharge")

style_date_axis(ax)

ax.legend()
st.pyplot(fig)


# INSIGHTS (CLEAN TEXT)

st.markdown("---")
st.subheader("Key Insights")

st.write(f"""
• Transfer Efficiency: {filtered['transfer_efficiency'].mean():.2f}
• Discharge Effectiveness: {filtered['discharge_effectiveness'].mean():.2f}
• Average Backlog: {int(filtered['backlog'].mean())}

If backlog stays positive → system delay
Improving discharge reduces pressure on system
""")


# DATA

with st.expander("View Data"):
```

```
    st.dataframe(filtered)

```